

Cybersecurity Vulnerability Assessment Report

Passive security audit for testaspnet.vulnweb.com



Executive Summary

This report details the findings of a passive vulnerability assessment conducted on testaspnet.vulnweb.com. A total of **12 security issues** were identified. Importantly, none of the identified vulnerabilities indicate a current compromise of the system. The most urgent finding is the **lack of HTTPS encryption**, which exposes sensitive data in transit. While no critical vulnerabilities were found, several medium and low-risk issues were identified that, if left unaddressed, could increase the attack surface.

Scope and Methodology

This assessment was strictly **passive and read-only**, meaning no active exploitation, login bypass attempts, or unauthorized access was performed. The objective was to identify potential security weaknesses through observation and analysis of publicly accessible information.

Tools Used:

- Nmap (for network and OS fingerprinting)
- OWASP ZAP (for passive scanning and traffic analysis)
- Chrome DevTools (for inspecting network traffic and client-side behavior)

Explicitly Excluded:

- Exploitation of identified vulnerabilities
- Login bypass or unauthorized access attempts
- Active probing that could impact system stability

Findings Summary

Finding Name	Risk Level
Missing HTTPS	High
Session Cookie Sent Unencrypted	High
Missing SameSite Cookie Attribute	Medium
Missing Anti-CSRF Tokens	Medium
Missing X-Frame-Options Header	Medium
Missing Content Security Policy (CSP) Header	Medium
Outdated ASP.NET Version Exposed (2.0.50727)	Medium
Missing X-Content-Type-Options Header	Low
IIS Server Version Disclosed	Low
OS Fingerprintable via Nmap	Low
Session Handling Flagged for Review	Low
Charset Mismatch	Low

Detailed Findings

High Risk

- **Missing HTTPS:**
 - **Issue:** The website does not use HTTPS for encrypted communication.
 - **Why it Matters:** All data transmitted between the user's browser and the server is sent in plain text, making it vulnerable to eavesdropping and man-in-the-middle attacks. This could expose sensitive information like login credentials, personal data, and session tokens.
 - **Recommended Fix:** Implement an SSL/TLS certificate and configure the web server to enforce HTTPS for all connections.
- **Session Cookie Sent Unencrypted:**
 - **Issue:** Session cookies are transmitted over an unencrypted HTTP connection.

- **Why it Matters:** This directly relates to the missing HTTPS issue. If a session cookie is intercepted, an attacker can hijack the user's session and impersonate them without needing to log in.
- **Recommended Fix:** Ensure all cookies, especially session cookies, are only sent over an HTTPS connection. Set the `Secure` flag on all cookies.

Medium Risk

- **Missing SameSite Cookie Attribute:**
 - **Issue:** The `SameSite` attribute is not specified for cookies.
 - **Why it Matters:** This attribute helps mitigate cross-site request forgery (CSRF) attacks by controlling when cookies are sent with cross-site requests. Without it, browsers may send cookies with potentially harmful cross-site requests.
 - **Recommended Fix:** Set the `SameSite` attribute to `Strict` or `Lax` for all relevant cookies.
- **Missing Anti-CSRF Tokens:**
 - **Issue:** No anti-Cross-Site Request Forgery (CSRF) tokens were detected in forms or requests that modify state.
 - **Why it Matters:** CSRF attacks trick a user's browser into performing unwanted actions on a web application they are authenticated to. Without CSRF tokens, the application is vulnerable to such attacks.
 - **Recommended Fix:** Implement and validate unique, unpredictable CSRF tokens for all state-changing requests.
- **Missing X-Frame-Options Header:**
 - **Issue:** The `X-Frame-Options` HTTP response header is not present.
 - **Why it Matters:** This header prevents clickjacking attacks, where an attacker embeds your site in an iframe on their malicious site to trick users into performing actions unintentionally. It controls whether your page can be rendered within a frame.
 - **Recommended Fix:** Add the `X-Frame-Options` header with an appropriate value (e.g., `deny` or `sameorigin`).
- **Missing Content Security Policy (CSP) Header:**
 - **Issue:** A `Content-Security-Policy` HTTP header is not configured.
 - **Why it Matters:** CSP is a powerful defense mechanism against various types of attacks, including cross-site scripting (XSS) and data injection. It allows you to specify which resources (scripts, styles, etc.) are allowed to load, significantly reducing the risk of malicious code execution.
 - **Recommended Fix:** Implement a robust CSP header tailored to the application's needs.
- **Outdated ASP.NET Version Exposed (2.0.50727):**
 - **Issue:** The web server reveals that it is running an old version of ASP.NET (2.0.50727).
 - **Why it Matters:** Older software versions often have known vulnerabilities that attackers can exploit. Running outdated frameworks increases the risk of compromise.
 - **Recommended Fix:** Update to the latest stable version of ASP.NET and ensure all dependencies are current. If upgrading is not immediately possible, implement additional security controls to mitigate known risks.

Low Risk

- **Missing X-Content-Type-Options Header:**
 - **Issue:** The `X-Content-Type-Options` HTTP response header is missing.

- **Why it Matters:** This header prevents MIME-sniffing attacks, where a browser might try to guess the content type of a response, potentially leading to the execution of malicious code if the server incorrectly identifies a file.
- **Recommended Fix:** Add the header with the value .
- **IIS Server Version Disclosed:**
 - **Issue:** The specific version of the IIS web server is revealed in HTTP headers.
 - **Why it Matters:** Knowing the exact server version can help attackers find specific exploits targeted at that version, even if the underlying OS is not fully identified.
 - **Recommended Fix:** Configure IIS to suppress or obfuscate the server banner in HTTP responses.
- **OS Fingerprintable via Nmap:**
 - **Issue:** The operating system running on the server could be identified through network analysis (e.g., Nmap's OS detection).
 - **Why it Matters:** Similar to server version disclosure, knowing the OS helps attackers tailor their attacks to known vulnerabilities specific to that operating system.
 - **Recommended Fix:** Implement network-level security controls and consider techniques to make OS fingerprinting more difficult, though this is often secondary to patching and configuration.
- **Session Handling Flagged for Review:**
 - **Issue:** Aspects of the session management mechanism warrant further review.
 - **Why it Matters:** Insecure session handling can lead to session hijacking, fixation, or prediction. This is a broad category that requires deeper investigation into how session IDs are generated, managed, and invalidated.
 - **Recommended Fix:** Conduct a thorough review of the application's session management implementation, ensuring secure generation, transmission (via HTTPS), and expiration of session tokens.
- **Charset Mismatch:**
 - **Issue:** A mismatch was observed between the declared character set and the actual character set of responses.
 - **Why it Matters:** This can lead to rendering issues or, in some cases, potential cross-site scripting (XSS) vulnerabilities if a browser interprets characters incorrectly.
 - **Recommended Fix:** Ensure that the header accurately reflects the character encoding of the response, typically UTF-8.

Prioritized Remediation Plan

High Priority

1. **Implement HTTPS:** Obtain and install an SSL/TLS certificate. Configure the web server to redirect all HTTP traffic to HTTPS. This is the *most critical* step to protect data in transit.
2. **Secure Session Cookies:** Ensure the flag is set on all session cookies. This will prevent them from being transmitted over unencrypted HTTP connections.

Medium Priority

1. **Implement Anti-CSRF Tokens:** Integrate CSRF tokens into all forms and state-changing requests.
2. **Configure Security Headers:** Add (e.g., or) and (CSP) headers. Develop a CSP tailored to the application's requirements.
3. **Update ASP.NET:** Upgrade to the latest supported version of ASP.NET and its components to address known vulnerabilities in older versions.
4. **Secure Cookie Attributes:** Implement the attribute for cookies (e.g., or).

Low Priority

1. **Configure Security Headers:** Add the header with the value .
2. **Obfuscate Server Banners:** Configure the IIS web server to suppress or anonymize the HTTP header to hide the version information.
3. **Review Session Handling:** Conduct a detailed review of the session management implementation to ensure robust security practices.
4. **Address Charset Mismatches:** Ensure consistency between the header and the actual character encoding of responses.

Conclusion

The vulnerability assessment of testaspnet.vulnweb.com identified 12 issues, with the most significant risk stemming from the **absence of HTTPS encryption**. This vulnerability, along with the unencrypted transmission of session cookies, creates a substantial risk of data interception and session hijacking. While no immediate compromise indicators were found, addressing these security weaknesses is crucial for protecting user data and maintaining the integrity of the application. Fortunately, all identified issues are addressable through configuration changes and standard web security best practices, making remediation a high priority for improving the overall security posture of the website.